In [ ]:
```
import marimo as mo
```

Trabalho Prático: Sudoku Genérico como CSP

Contexto

O Sudoku clássico — uma grelha $n^2 \times n^2$ onde cada linha, cada coluna e cada bloco $n \times n$ tem de conter todos os valores de 1 a n^2 sem repetições — é um exemplo canónico de **problema de satisfação de restrições (CSP)**: a "regra" é sempre a mesma (um conjunto de células tem de ter valores todos diferentes), o que muda de linha para linha, de coluna para coluna e de bloco para bloco é apenas **que células pertencem a esse conjunto**.

Isso sugere uma abstração única — um grupo de células com a restrição "todos diferentes", opcionalmente com algumas células já fixas a um valor — a partir da qual linhas, colunas, blocos e ainda outras variantes de Sudoku (diagonais, regiões irregulares, grelhas sobrepostas, etc.) podem ser todas construídas sem duplicar lógica de restrição nenhuma.

Este é um problema de **modelação e resolução de CSP**. Cabe-te a ti escolher a técnica de resolução e justificá-la — o enunciado não fornece código de modelação nem de apresentação de resultados, apenas a interface que o teu notebook tem de expor (secção seguinte) para poder ser testado automaticamente.

Objetivo

Construir, num notebook Marimo, um gerador/resolvedor de Sudoku $n^2 \times n^2$ (com n parametrizável, tipicamente $n = 3$) que:

1. representa qualquer **grupo de células com restrição "todos diferentes"** através de uma classe genérica (secção "`box` — grupo genérico de células"),
2. constrói **linhas, colunas e blocos** como casos particulares dessa classe genérica — os blocos através de uma especialização dedicada a blocos $n \times n$, as linhas e colunas através de uma especialização dedicada a sequências retas de células (secção "`cube` e `path`"),
3. gera **aleatoriamente** um subconjunto de células já preenchidas (as "pistas" iniciais do puzzle), usando a mesma abstração genérica (secção "Geração aleatória de pistas"),
4. monta o modelo completo (linhas + colunas + blocos + pistas) e o resolve como CSP, devolvendo a grelha preenchida ou sinalizando que não há solução (secção "Resolução").

Requisitos obrigatórios

O teu notebook tem de expor, com este comportamento, os seguintes elementos (os nomes propostos abaixo são sugestões que facilitam a correção automática — podes usar outros, desde que documentes a correspondência):

`box` — grupo genérico de células (R1)

Uma classe que representa **qualquer** conjunto de células da grelha às quais se aplica a restrição "todos os valores diferentes", com algumas delas possivelmente já fixas:

- guarda internamente uma associação `(linha, coluna) → valor ou None` (`None` = célula livre; um inteiro = célula fixa/pinada a esse valor);
- um construtor que aceita opcionalmente esse conjunto inicial de células (vazio por omissão);
- um método `add(i, j, val=None)` que acrescenta a célula `(i, j)` ao grupo, opcionalmente fixando-a a `val`, e que **rejeita** (levanta exceção) coordenadas fora da grelha ou valores fora do intervalo $[1, n^2]$;
- uma forma de obter a representação do grupo como matriz $n^2 \times n^2$, com zeros nas células não pertencentes ao grupo ou não fixas, e o valor fixo nas restantes.

Esta classe **não deve saber nada** sobre linhas, colunas, blocos ou Sudoku — só sabe lidar com "um conjunto de células, algumas fixas". Essa generalidade é o que te vai permitir, mais tarde, tratar da mesma forma linhas, colunas, blocos, pistas aleatórias e (nas extensões opcionais) diagonais ou regiões irregulares.

`cube` e `path` — duas formas concretas de grupo (R2, R3)

A partir da classe genérica, define duas especializações:

- **R2.** Um grupo que representa o **bloco** $n \times n$ cujo canto superior esquerdo é a célula $(i \cdot n,\ j \cdot n)$, parametrizado pelos índices de bloco (i, j) com $0 \leq i, j < n$.
- **R3.** Um grupo que representa o **troço reto** (horizontal ou vertical) de células entre duas coordenadas `inicio` e `fim`, inclusive — tem de funcionar tanto para `fim` "depois" de `inicio` como "antes" (ou seja, percorrer a sequência em qualquer sentido).

Geração aleatória de pistas (R4)

Uma função que devolve um grupo (`box`) com k células escolhidas aleatoriamente na grelha, cada uma fixa a um valor também escolhido aleatoriamente em $[1, n^2]$ (k deve ter um valor por omissão razoável, por exemplo da ordem de n). Repara que esta função **não precisa de nenhuma classe nova** — o resultado é, de novo, apenas um `box`.

Modelo e resolução (R5, R6)

- **R5.** Um modelo de CSP para a grelha $n^2 \times n^2$, com uma variável inteira por célula, cada uma no intervalo $[1, n^2]$; um método que recebe **um número arbitrário de grupos** (`box` , `cube` , `path` , ou pistas aleatórias — o modelo não deve distinguir a sua origem) e, para cada um, impõe que as suas células sejam todas diferentes e fixa as que tiverem valor atribuído; e um método de resolução que devolve a grelha preenchida ou sinaliza, de forma distinguível, que o puzzle não tem solução.
- **R6.** Um Sudoku $n^2 \times n^2$ completo é montado juntando: todas as linhas, todas as colunas, todos os blocos $n \times n$ e (pelo menos) um grupo de pistas aleatórias — e resolvido.

Como testar/validar

O teu notebook (ou um ficheiro de testes à parte) tem de verificar automaticamente, para uma grelha resolvida:

- que cada linha, cada coluna e cada bloco $n \times n$ contém exatamente os valores $1 \ldots n^2$, sem repetições;
- que as células fixadas pelas pistas aleatórias mantêm, na solução, o valor com que foram fixadas;
- que `add` (ou equivalente) rejeita coordenadas fora da grelha e valores fora de $[1, n^2]$.

Corre o fluxo completo (gerar pistas aleatórias → montar linhas + colunas + blocos + pistas → resolver → validar) pelo menos uma vez com $n = 3$ (Sudoku clássico 9×9) e confirma que também funciona com outro valor de n (ex.: $n = 2$, grelha 4×4), para garantires que nada está fixo a 9×9 no teu código.

O que é deixado ao teu critério

O enunciado define **que abstrações** o notebook tem de expor e **que comportamento** têm de ter, não **como** as deves implementar. Ficam ao teu critério, desde que justificadas no notebook:

- a técnica e biblioteca de resolução do CSP (CP-SAT do OR-Tools é a sugestão da disciplina, mas és livre de escolher outra abordagem de Lógica Computacional, justificando a escolha);
- a estrutura de dados interna do grupo genérico (dicionário, matriz esparsa, etc.);
- a forma de apresentar a grelha resultante (texto, tabela, `mo.ui` , gráfico — o que achares mais claro);
- o comportamento exato quando o puzzle gerado aleatoriamente não tem solução (podes, por exemplo, tentar novas pistas aleatórias até obteres um puzzle solúvel, ou simplesmente reportar o insucesso — justifica a escolha).

Extensões opcionais (bónus)

A generalidade do `box` é o que torna estas extensões possíveis sem tocar no modelo CSP em si — cada uma acrescenta apenas **novos grupos** de células:

- **Sudoku diagonal (X-Sudoku)**: acrescenta um grupo (`box`, sem precisar de nova subclasse) para cada uma das duas diagonais principais, também elas restritas a "todos diferentes".
- **Sudoku irregular (jigsaw)**: substitui os blocos $n \times n$ regulares por regiões de forma arbitrária mas do mesmo tamanho, cada uma representada como um `box` construído célula a célula em vez de por `cube`.
- **Hyper-Sudoku / Windoku**: acrescenta 4 blocos extra (também `box`, de forma semelhante a `cube` mas sem estarem alinhados com a grelha $n \times n$ de blocos) sobrepostos aos existentes.
- **Escala**: mostra que o teu código funciona (talvez mais devagar) para $n = 6$ (grelha 36×36) sem alterações, e discute os limites de desempenho que encontraste.
- **Sudoku tridimensional** define a estrutura de "boxes" numa grelha $n^2 \times n^2 \times n^2$.

TP2 - Sudoku Genérico

Introdução

O Sudoku pode ser modelado como um Problema de Satisfação de Restrições (CSP). Numa grelha `n² × n²`, cada célula deve assumir um valor entre `1` e `n²`, sendo necessário garantir que não existem valores repetidos em cada linha, coluna e bloco `n × n`.

O objetivo deste trabalho é construir uma implementação genérica, parametrizada por `n`, que permita representar estes diferentes grupos de células através da mesma abstração e resolver o problema utilizando programação com restrições.

Abordagem

A abordagem utilizada baseia-se numa abstração genérica denominada `box`, que representa um conjunto de células às quais é aplicada a restrição de valores todos diferentes.

A partir desta abstração são construídos:

- `cube`, para representar blocos `n × n`;
- `path`, para representar linhas e colunas;
- grupos de pistas, contendo células com valores previamente fixados.

O modelo CSP cria uma variável inteira para cada célula da grelha e utiliza o CP-SAT do OR-Tools para impor as restrições `AllDifferent` e os valores fixados pelas pistas.

Desta forma, o solver não necessita de distinguir se um grupo representa uma linha, uma coluna, um bloco ou outro tipo de região.

R1 - Classe `box`

A classe `box` representa um grupo genérico de células de uma grelha de Sudoku.

Cada célula é identificada pelas suas coordenadas `(linha, coluna)` e pode:

- não ter um valor fixo (`None`);
- ter um valor previamente fixado.

A classe não distingue linhas, colunas ou blocos. Apenas guarda um conjunto de células às quais, mais tarde, poderá ser aplicada a restrição de valores todos diferentes.

Para representar internamente as células será utilizado um dicionário da forma:

`(linha, coluna) -> valor ou none`

A dimensão da grelha é `n² × n²` , permitindo utilizar a mesma implementação para diferentes tamanhos de Sudoku.

In [ ]:
```python
class box:
    def __init__(self, n, cells=None):
        self.n = n
        self.size = n ** 2
        self.cells = {}

        if cells is not None:
            for (i, j), val in cells.items():
                self.add(i, j, val)

    def add(self, i, j, val=None):
        # Verificar se as coordenadas pertencem à grelha
        if i < 0 or i >= self.size or j < 0 or j >= self.size:
            raise ValueError("Coordenadas fora da grelha")

        # Verificar se o valor é válido
        if val is not None and (val < 1 or val > self.size):
            raise ValueError("Valor fora do intervalo permitido")

        self.cells[(i, j)] = val

    def matrix(self):
        matriz = [
            [0 for _ in range(self.size)]
            for _ in range(self.size)
        ]
```

```
        for (i, j), val in self.cells.items():
            if val is not None:
                matriz[i][j] = val

        return matriz
```

Testes da classe `box`

São realizados alguns testes simples, para verificar:

- adição de células livres;
- adição de células com valores fixos;
- representação matricial;
- rejeição de coordenadas e valores inválidos.

In [ ]:
```
b = box(3)

b.add(0, 0, 5)
b.add(0, 1)
b.add(2, 3, 7)

print(b.cells)
```

In [ ]:
```
matriz = b.matrix()

for linha in matriz:
    print(linha)
```

In [ ]:
```
try:
    b.add(9, 0)
except ValueError:
    print("Coordenada inválida corretamente rejeitada.")
else:
    raise RuntimeError("Uma coordenada inválida foi aceite.")

try:
    b.add(0, 0, 10)
except ValueError:
    print("Valor inválido corretamente rejeitado.")
else:
    raise RuntimeError("Um valor inválido foi aceite.")
```

R2 - Classe `cube`

A classe `cube` é uma especialização da classe `box` e representa um bloco quadrado `n × n` da grelha.

Cada bloco é identificado pelos índices `(i, j)`, com `0 <= i, j < n`.

O canto superior esquerdo do bloco encontra-se na posição:

`(i * n, j * n)`

A partir dessa posição são adicionadas ao grupo todas as `n²` células que constituem o bloco.

In [ ]:
```python
class cube(box):
    def __init__(self, n, i, j):
        super().__init__(n)

        # Verificar se os índices do bloco são válidos
        if i < 0 or i >= n or j < 0 or j >= n:
            raise ValueError("Índices de bloco fora do intervalo permitido")

        linha_inicial = i * n
        coluna_inicial = j * n

        for linha in range(linha_inicial, linha_inicial + n):
            for coluna in range(coluna_inicial, coluna_inicial + n):
                self.add(linha, coluna)
```

Testes da classe `cube`

É verificado que:

- o bloco contém exatamente `n²` células;
- as coordenadas correspondem ao bloco esperado;
- índices de bloco inválidos são rejeitados.

In [ ]:
```python
c = cube(3, 1, 2)

print(c.cells)
print("Número de células:", len(c.cells))
```

In [ ]:
```python
try:
    cube(3, 3, 0)
    assert False, "Devia ter sido lançada uma exceção"
except ValueError:
    print("Índice inválido corretamente rejeitado.")
```

R3 - Classe `path`

A classe `path` é uma especialização da classe `box` que representa um troço reto de células da grelha.

O troço pode ser:

- horizontal, quando a linha inicial e final são iguais;

- vertical, quando a coluna inicial e final são iguais.

As coordenadas `inicio` e `fim` estão incluídas no grupo e o percurso deve funcionar nos dois sentidos.

In [ ]:
```python
class path(box):
    def __init__(self, n, inicio, fim):
        super().__init__(n)

        linha_inicio, coluna_inicio = inicio
        linha_fim, coluna_fim = fim

        # Verificar se as coordenadas pertencem à grelha
        if (linha_inicio < 0 or linha_inicio >= self.size or
            coluna_inicio < 0 or coluna_inicio >= self.size or
            linha_fim < 0 or linha_fim >= self.size or
            coluna_fim < 0 or coluna_fim >= self.size):
            raise ValueError("Coordenadas fora da grelha")

        # Caminho horizontal
        if linha_inicio == linha_fim:
            passo = 1 if coluna_fim >= coluna_inicio else -1

            for coluna in range(
                coluna_inicio,
                coluna_fim + passo,
                passo
            ):
                self.add(linha_inicio, coluna)

        # Caminho vertical
        elif coluna_inicio == coluna_fim:
            passo = 1 if linha_fim >= linha_inicio else -1

            for linha in range(
                linha_inicio,
                linha_fim + passo,
                passo
            ):
                self.add(linha, coluna_inicio)

        else:
            raise ValueError("O path tem de ser horizontal ou vertical")
```

Testes da classe `path`

São testados:

- um caminho horizontal;
- um caminho vertical;
- um percurso no sentido inverso;
- a rejeição de um caminho diagonal.

In [ ]:
```
p_horizontal = path(3, (0, 0), (0, 8))

print(p_horizontal.cells)
```

In [ ]:
```
p_vertical = path(3, (0, 8), (8, 8))

print(p_vertical.cells)
```

In [ ]:
```
p_inverso = path(3, (8, 8), (0, 8))

print(p_inverso.cells)
```

In [ ]:
```
try:
    path(3, (0, 0), (3, 3))
    assert False, "Devia ter sido lançada uma exceção"
except ValueError:
    print("Path diagonal corretamente rejeitado.")
```

R4 - Geração aleatória de pistas

As pistas iniciais do Sudoku são representadas através da classe `box`.

A função escolhe `k` posições distintas da grelha e associa a cada uma um valor aleatório entre `1` e `n²`.

Não é necessária uma nova classe, uma vez que uma pista é apenas uma célula pertencente a um grupo com um valor já fixado.

In [ ]:
```
import random
```

In [ ]:
```
def gerar_pistas(n, k=None):
    tamanho = n ** 2

    # Por omissão são geradas n pistas
    if k is None:
        k = n

    total_celulas = tamanho ** 2

    if k < 0 or k > total_celulas:
        raise ValueError("Número de pistas inválido")

    # Todas as coordenadas possíveis da grelha
    coordenadas = [
        (linha, coluna)
        for linha in range(tamanho)
        for coluna in range(tamanho)
    ]

    # Escolher k células diferentes
    escolhidas = random.sample(coordenadas, k)
```

```
    pistas = box(n)

    for linha, coluna in escolhidas:
        valor = random.randint(1, tamanho)
        pistas.add(linha, coluna, valor)

    return pistas
```

Teste da função `gerar_pistas`

É verificado que:

- gerar pistas devolve valores aleatórios
- valor por omissão é razoável

In [ ]:
```
pistas = gerar_pistas(3, 5)

print(pistas.cells)
```

In [ ]:
```
pistas_default = gerar_pistas(3)

assert len(pistas_default.cells) == 3

print("Número de pistas por omissão correto.")
```

R5 - Modelo CSP

O Sudoku é modelado como um problema de satisfação de restrições.

Para cada célula da grelha é criada uma variável inteira com domínio entre `1` e `n²`.

O modelo recebe grupos de células independentemente da sua origem. Para cada grupo é aplicada a restrição `AllDifferent`, garantindo que todas as suas células têm valores distintos.

Quando uma célula do grupo já possui um valor fixado, é também adicionada uma restrição que obriga a variável dessa célula a assumir esse valor.

In [ ]:
```
from ortools.sat.python import cp_model
```

In [ ]:
```
class sudoku_csp:
    def __init__(self, n):
        self.n = n
        self.size = n ** 2

        self.model = cp_model.CpModel()

        self.variables = {}
```

```
        for linha in range(self.size):
            for coluna in range(self.size):
                self.variables[(linha, coluna)] = self.model.NewIntVar(
                    1,
                    self.size,
                    f"x_{linha}_{coluna}"
                )
    def add_groups(self, *groups):
        for group in groups:
            variables_group = []

            for (linha, coluna), valor in group.cells.items():
                var = self.variables[(linha, coluna)]

                variables_group.append(var)

                if valor is not None:
                    self.model.Add(var == valor)

            if len(variables_group) > 1:
                self.model.AddAllDifferent(variables_group)
    def solve(self):
        solver = cp_model.CpSolver()

        status = solver.Solve(self.model)
        status_name = solver.StatusName(status)

        if status_name == "OPTIMAL" or status_name == "FEASIBLE":
            solucao = [
                [
                    solver.Value(self.variables[(linha, coluna)])
                    for coluna in range(self.size)
                ]
                for linha in range(self.size)
            ]

            return solucao

        return None
```

Teste do modelo CSP

Para testar o modelo isoladamente é utilizada uma grelha `4 × 4`.

É criado um grupo correspondente à primeira linha, sendo fixado o valor `1` na célula `(0, 0)`.

O modelo deve encontrar valores entre `1` e `4` para as quatro células da linha, mantendo a pista e garantindo que todos os valores são diferentes.

In [ ]:
```
teste_grupo = path(2, (0, 0), (0, 3))

# Fixar uma das células do grupo
teste_grupo.add(0, 0, 1)
```

```
teste_modelo = sudoku_csp(2)
teste_modelo.add_groups(teste_grupo)

teste_solucao = teste_modelo.solve()

print(teste_solucao)
```

In [ ]:
```
grupo_impossivel = path(2, (0, 0), (0, 3))

grupo_impossivel.add(0, 0, 1)
grupo_impossivel.add(0, 1, 1)

modelo_impossivel = sudoku_csp(2)
modelo_impossivel.add_groups(grupo_impossivel)

solucao_impossivel = modelo_impossivel.solve()

assert solucao_impossivel is None

print("Puzzle impossível corretamente detetado.")
```

R6 - Construção e resolução do Sudoku completo

Um Sudoku completo é construído através dos grupos definidos anteriormente:

- `n²` grupos `path` para as linhas;
- `n²` grupos `path` para as colunas;
- `n²` grupos `cube` para os blocos;
- um grupo `box` contendo pistas aleatórias.

Todos estes grupos são adicionados ao mesmo modelo CSP. Desta forma, a mesma restrição `AllDifferent` é aplicada uniformemente a linhas, colunas e blocos.

In [ ]:
```
def resolver_sudoku(n, k=None):
    size = n ** 2

    # Criar todas as linhas
    linhas_sudoku = [
        path(n, (linha, 0), (linha, size - 1))
        for linha in range(size)
    ]

    # Criar todas as colunas
    colunas_sudoku = [
        path(n, (0, coluna), (size - 1, coluna))
        for coluna in range(size)
    ]

    # Criar todos os blocos n x n
    blocos_sudoku = [
        cube(n, i, j)
```

```
        for i in range(n)
        for j in range(n)
    ]

    # Gerar pistas aleatórias
    pistas_sudoku = gerar_pistas(n, k)

    # Criar o modelo CSP
    modelo_sudoku = sudoku_csp(n)

    # Adicionar todos os grupos ao modelo
    modelo_sudoku.add_groups(
        *linhas_sudoku,
        *colunas_sudoku,
        *blocos_sudoku,
        pistas_sudoku
    )

    # Resolver
    solucao_sudoku = modelo_sudoku.solve()

    return solucao_sudoku, pistas_sudoku
```

Validação automática

Depois de resolver o Sudoku, a solução é validada automaticamente.

A validação verifica que:

- cada linha contém exatamente os valores de `1` a `n²`;
- cada coluna contém exatamente os valores de `1` a `n²`;
- cada bloco `n × n` contém exatamente os valores de `1` a `n²`;
- todas as pistas iniciais são respeitadas.

In [ ]:
```
def validar_sudoku(solucao, n, pistas):
    if solucao is None:
        return False

    size = n ** 2
    valores_esperados = set(range(1, size + 1))

    # Verificar dimensões da grelha
    if len(solucao) != size:
        return False

    for linha in solucao:
        if len(linha) != size:
            return False

    # Verificar linhas
    for linha in range(size):
        if set(solucao[linha]) != valores_esperados:
            return False
```

```
    # Verificar colunas
    for coluna in range(size):
        valores_coluna = {
            solucao[linha][coluna]
            for linha in range(size)
        }

        if valores_coluna != valores_esperados:
            return False

    # Verificar blocos n x n
    for bloco_i in range(n):
        for bloco_j in range(n):
            valores_bloco = set()

            linha_inicial = bloco_i * n
            coluna_inicial = bloco_j * n

            for linha in range(linha_inicial, linha_inicial + n):
                for coluna in range(coluna_inicial, coluna_inicial + n):
                    valores_bloco.add(solucao[linha][coluna])

            if valores_bloco != valores_esperados:
                return False

    # Verificar pistas
    for (linha, coluna), valor in pistas.cells.items():
        if valor is not None:
            if solucao[linha][coluna] != valor:
                return False

    return True
```

O fluxo completo é testado para `n = 3` e `n = 2`.

In [ ]:

```
solucao_r6, pistas_r6 = resolver_sudoku(3, 5)

print("Pistas:")
print(pistas_r6.cells)

print("\nSolução:")

if solucao_r6 is not None:
    for _linha_r6 in solucao_r6:
        print(_linha_r6)

    print("\nValidação:")

    if validar_sudoku(solucao_r6, 3, pistas_r6):
        print("Sudoku válido.")
    else:
        print("A solução não é válida.")
```

```
else:
    print("O conjunto de pistas gerado não tem solução.")
```

In [ ]:
```
solucao__r6, pistas__r6 = resolver_sudoku(2, 3)

print("Pistas:")
print(pistas__r6.cells)

print("\nSolução:")

if solucao__r6 is not None:
    for _linha__r6 in solucao__r6:
        print(_linha__r6)

    print("\nValidação:")

    if validar_sudoku(solucao__r6, 2, pistas__r6):
        print("Sudoku válido.")
    else:
        print("A solução não é válida.")

else:
    print("O conjunto de pistas gerado não tem solução.")
```

Decisões de implementação

Solver

Foi utilizado o CP-SAT da biblioteca OR-Tools. Esta escolha permite representar diretamente as células do Sudoku como variáveis inteiras com domínio finito e utilizar a restrição `AllDifferent`, adequada às regras de linhas, colunas e blocos do Sudoku.

Representação dos grupos

Cada `box` utiliza um dicionário da forma:

`(linha, coluna) -> valor ou None`

Esta representação permite distinguir células pertencentes ao grupo sem valor fixado (`None`) de células com uma pista previamente definida.

Parametrização

A implementação não assume uma grelha fixa `9 × 9`. A dimensão é sempre calculada através de `n²`, permitindo utilizar o mesmo código para, por exemplo, `n=2`, `n=3` ou valores superiores.

Pistas aleatórias

As posições das pistas são escolhidas sem repetição e os seus valores são gerados aleatoriamente entre `1` e `n²`.

Como esta geração pode produzir pistas incompatíveis entre si, o solver pode concluir que não existe solução. Nesse caso, a função de resolução devolve `None` e o notebook informa que o conjunto de pistas gerado não tem solução.

Apresentação

A solução é apresentada em formato textual, linha a linha. Esta forma é suficiente para observar diretamente a grelha produzida e simplifica a demonstração do funcionamento do modelo.

Testes de correção

A correção da implementação é testada em diferentes níveis:

- validação de coordenadas e valores na classe `box`;
- construção de blocos através de `cube`;
- construção de percursos horizontais e verticais através de `path`;
- deteção de um CSP impossível;
- validação automática de linhas, colunas, blocos e pistas;
- execução completa para `n=3`;
- execução completa para `n=2`.

A utilização de dois valores diferentes de `n` permite ainda verificar que a implementação não contém dimensões fixas específicas de um Sudoku `9 × 9`.

In [ ]:
```
import time
```

In [ ]:
```
def medir_tempo(n, k=0):
    inicio = time.perf_counter()

    solucao, pistas = resolver_sudoku(n, k)

    fim = time.perf_counter()

    return fim - inicio, solucao is not None
```

In [ ]:
```python
for _n_escala in [2, 3, 4]:
    _tempo, _tem_solucao = medir_tempo(_n_escala, 0)

    print(
        f"n={_n_escala} | "
        f"grelha={_n_escala**2}x{_n_escala**2} | "
        f"tempo={_tempo:.4f}s | "
        f"solução={_tem_solucao}"
    )
```

Eficiência computacional e escalabilidade

Para avaliar a eficiência da implementação foi medido o tempo necessário para construir e resolver instâncias com diferentes valores de `n`.

Neste teste são utilizadas zero pistas aleatórias, de forma a comparar instâncias equivalentes e evitar que a aleatoriedade origine problemas com graus de dificuldade diferentes ou instâncias sem solução.

A dimensão da grelha é `n² × n²`, pelo que o número de variáveis cresce com `n⁴`:

- `n=2`: 16 variáveis;
- `n=3`: 81 variáveis;
- `n=4`: 256 variáveis.

Os resultados mostram um aumento do tempo de resolução à medida que a dimensão do problema aumenta. Para `n=2`, o tempo médio foi de cerca de 0.016 s, enquanto para o Sudoku clássico (`n=3`) aumentou para cerca de 0.059 s. Para `n=4`, correspondente a uma grelha `16 × 16`, o tempo médio foi de aproximadamente 0.324 s.

Apesar do aumento do custo computacional, todas as instâncias foram resolvidas em menos de um segundo e sem qualquer alteração ao modelo. Isto mostra que a implementação é efetivamente parametrizada e consegue lidar com problemas maiores do que o Sudoku clássico.

Resultados

`n`	Dimensão da grelha	Execução 1	Execução 2	Execução 3	Tempo médio
2	4 × 4	0.0145 s	0.0208 s	0.0137 s	**0.0163 s**
3	9 × 9	0.0478 s	0.0656 s	0.0631 s	**0.0588 s**
4	16 × 16	0.3328 s	0.3198 s	0.3201 s	**0.3242 s**

Contribuição da LLM

Durante o desenvolvimento foi utilizada uma ferramenta LLM como apoio à compreensão do enunciado, desenvolvimento e depuração da solução.

A utilização da LLM incidiu principalmente nas seguintes tarefas:

- interpretação dos requisitos R1–R6;
- esclarecimento da abstração `box` e da diferença entre células livres, células fixadas e células não pertencentes a um grupo;
- apoio na implementação das classes `cube` e `path`;
- discussão da modelação do Sudoku como CSP utilizando CP-SAT;
- explicação da utilização de `AllDifferent`;
- proposta e revisão de testes de correção;
- identificação e resolução de problemas específicos do Marimo;
- adaptação do código às diferenças da versão instalada do OR-Tools;
- apoio na organização do notebook e na definição dos testes de escalabilidade.

As sugestões produzidas pela LLM foram revistas antes de serem integradas. O código final foi executado e validado no ambiente local, sendo os resultados apresentados no próprio notebook.

Exemplos de interação

Questão: Qual é a diferença entre uma célula com valor `None` numa `box` e uma célula que não pertence à `box`?

Contributo: A discussão permitiu clarificar que `None` representa uma célula pertencente ao grupo mas ainda livre, enquanto uma célula ausente não participa nas restrições desse grupo.

Problema: A versão instalada do OR-Tools não disponibilizava `cp_model.OPTIMAL`.

Contributo: A depuração levou à utilização de `solver.StatusName(status)` para distinguir soluções válidas de casos sem solução.

Conclusão

Neste trabalho foi desenvolvido um resolvedor genérico de Sudoku através de programação com restrições.

A abstração `box` permitiu representar de forma uniforme linhas, colunas, blocos e pistas, possibilitando a aplicação da mesma restrição `AllDifferent` a diferentes tipos de grupos.

O modelo foi implementado com CP-SAT e validado automaticamente em grelhas `4 × 4` e `9 × 9`. Foram também realizados testes de desempenho com dimensões superiores,

permitindo observar o aumento do custo computacional à medida que cresce o número de variáveis e restrições.

A abordagem adotada mostrou-se adequada para representar o Sudoku como CSP e mantém-se reutilizável para outras dimensões sem alterações à estrutura principal do modelo.

Registo da interação com a LLM